Breaking Language into Pieces: Understanding Tokenizers in LLMs

What Are Tokenizers?

A **tokenizer** is a tool that converts human-readable text into smaller units called **tokens** so that a machine learning model can process the text. Large Language Models (LLMs) and Transformer models cannot directly work with words and sentences in their original text form. Neural networks perform mathematical operations on numbers, so text must first be converted into a numerical representation. Tokenization is the first major step in this process.

A **token** is a small unit of text recognized by the tokenizer. A token can be a complete word, part of a word, a character, punctuation mark, or another text unit depending on the tokenizer being used. For example, the word **"learning"** might be represented as one token by one tokenizer but divided into smaller pieces such as **"learn" + "ing"** by another tokenizer.

After creating tokens, the tokenizer converts each token into a unique number called a **token ID**. These token IDs are what the Transformer model actually receives as input.

For example, consider the sentence:

"I love machine learning."

A simplified tokenization process could look like:

Text:

"I love machine learning."

↓

Tokens:

["I", "love", "machine", "learning", "."]

↓

Token IDs:

[45, 928, 3412, 6821, 13]

The actual tokens and token IDs depend entirely on the tokenizer being used.

The model's embedding layer then converts these token IDs into numerical vectors called **embeddings**, which can be processed by Transformer layers.

Therefore, the basic relationship is:

Text → Tokenizer → Tokens → Token IDs → Embeddings → Transformer Model

In simple terms, the tokenizer acts as a **bridge between human language and the numerical world of a neural network**.

How Do Tokenizers Work?

A tokenizer generally works through a series of steps. The exact process depends on the tokenizer, but the overall idea remains similar.

The first step is receiving the **raw text**. Suppose the user enters:

"Transformers are powerful!"

The tokenizer receives this text and may first perform **normalization**. Normalization means preparing text in a consistent format. Depending on the tokenizer, this could involve handling Unicode characters, spaces, capitalization, accents, or other formatting differences. Importantly, not every tokenizer performs the same normalization, so we should not assume that all tokenizers automatically convert text to lowercase or remove punctuation.

The next step is **pre-tokenization** or initial splitting. The tokenizer identifies possible boundaries in the text, such as spaces, punctuation marks, or other patterns. The exact rules depend on the tokenization algorithm.

The tokenizer then applies its main tokenization method. For example, a subword tokenizer may split an uncommon word into smaller pieces.

Consider:

"unbelievable"

Instead of requiring the entire word to exist in its vocabulary, a tokenizer might produce something conceptually similar to:

"un" + "believ" + "able"

These smaller pieces can be reused in many different words.

After the text has been divided into tokens, each token is looked up in the tokenizer's **vocabulary**. The vocabulary is a collection of tokens known by that tokenizer. Every token has an associated numerical ID.

For example:

"I" → 45

"love" → 928

"machine" → 3412

"learning" → 6821

The tokenizer therefore converts a sequence of tokens into a sequence of integers called **input IDs** or **token IDs**.

The process can be summarized as:

Raw Text → Normalize → Split/Tokenize → Tokens → Vocabulary Lookup → Token IDs

When using Hugging Face Transformers, you will frequently see these token IDs represented as **input_ids**.

A tokenizer can also create additional information required by the model. One important example is the **attention mask**. The attention mask helps indicate which positions contain actual input and which positions may contain padding.

Therefore, a Hugging Face tokenizer may conceptually produce something like:

Text → Tokenizer → input_ids + attention_mask

The model then uses these inputs for processing.

Encoding and Decoding

Tokenizers perform two closely related operations called **encoding** and **decoding**.

Encoding means converting text into token IDs.

For example:

"I love AI"

↓

Tokenizer

↓

[40, 1842, 9552]

These token IDs can then be passed to the Transformer model.

Decoding performs the reverse operation. It converts token IDs back into readable text.

For example:

[40, 1842, 9552]

↓

Tokenizer Decode

↓

"I love AI"

This becomes especially important during text generation. A language model predicts numerical token IDs rather than directly producing text. The tokenizer converts those generated token IDs back into text that humans can read.

The complete LLM input-output process can therefore be understood as:

Human Text → Tokenizer Encoding → Token IDs → Transformer Model → Predicted Token IDs → Tokenizer Decoding → Human-Readable Text

Different Types of Tokenizers

There are several approaches to tokenization. The most important ones to understand for Transformer and Hugging Face study are **word-level, character-level, subword, and byte-based tokenization**.

1. Word-Level Tokenization

A **word-level tokenizer** divides text mainly into complete words.

For example:

"I love machine learning."

could become:

["I", "love", "machine", "learning", "."]

This approach is simple and easy to understand because most tokens correspond directly to words.

However, word-level tokenization has a major problem: human languages contain enormous numbers of possible words. New names, technical terms, spelling variations, and uncommon words constantly appear.

Suppose the tokenizer knows the word:

"play"

but does not contain:

"playfulness"

in its vocabulary.

A pure word-level tokenizer may have to represent the unfamiliar word using a special **unknown token**, often written as [UNK].

To reduce unknown words, we could create a very large vocabulary, but this would increase memory requirements and make the system inefficient.

For this reason, pure word-level tokenization is generally less suitable for modern LLMs.

2. Character-Level Tokenization

A **character-level tokenizer** breaks text into individual characters.

For example:

"AI"

becomes:

["A", "I"]

and:

"cat"

becomes:

["c", "a", "t"]

Character-level tokenization has the advantage of requiring a relatively small vocabulary because individual characters can be combined to construct many words.

It also reduces the traditional unknown-word problem because an unfamiliar word can still be represented using its individual characters.

However, character-level tokenization creates much longer sequences.

For example:

"Transformer"

is one word, but character-level tokenization could require eleven character tokens.

Longer token sequences increase the amount of work the Transformer must perform. This makes pure character-level tokenization less efficient for many LLM applications.

3. Subword Tokenization

Subword tokenization provides a balance between word-level and character-level tokenization and is extremely important in modern NLP.

Instead of always treating an entire word as one token, the tokenizer can divide uncommon or complex words into smaller reusable pieces.

For example:

"unbelievable"

might conceptually become:

["un", "believ", "able"]

while a very common word such as:

"the"

might remain a single token.

The main advantage is that the tokenizer does not need every possible word in its vocabulary. New or rare words can often be constructed from smaller pieces that already exist.

For example, words such as:

"playing"

"played"

"player"

may reuse pieces related to:

"play"

This makes subword tokenization efficient because common words can remain compact while uncommon words can still be represented using smaller units.

Many Transformer models use subword or closely related byte-based tokenization approaches.

Important Subword Tokenization Algorithms

Byte-Pair Encoding (BPE)

Byte-Pair Encoding, or BPE, is a popular tokenization algorithm used to create subword vocabularies.

The basic idea is to begin with small units and repeatedly combine frequently occurring neighboring units.

Suppose a tokenizer frequently encounters combinations such as:

"l" + "o"

It may learn to combine them into:

"lo"

If:

"lo" + "w"

also appears frequently, it may later create:

"low"

Through repeated merging, common patterns become larger tokens while uncommon words can still be represented using smaller pieces.

The important idea to remember is:

BPE builds larger tokens by repeatedly merging frequent smaller token pairs.

WordPiece

WordPiece is another subword tokenization algorithm and is strongly associated with models such as BERT.

For example, WordPiece might represent a word conceptually as:

"play" + "##ing"

Here, **##ing** indicates that the token continues the previous word in the common BERT WordPiece representation.

Like BPE, WordPiece tries to create a useful vocabulary of complete words and smaller word pieces so that uncommon words can still be represented.

The main idea to remember is:

WordPiece breaks uncommon words into known subword pieces and is commonly associated with BERT-style models.

Unigram Tokenization

Unigram is another subword tokenization method. Instead of starting small and repeatedly merging pieces like BPE, Unigram begins with many possible token pieces and gradually removes less useful ones while learning probabilities for the remaining pieces.

A word can sometimes have several possible tokenizations.

For example:

"transformer"

might theoretically be represented as:

["transform", "er"]

or:

["trans", "form", "er"]

The Unigram model uses learned probabilities to determine suitable tokenizations.

The important idea is:

BPE mainly builds a vocabulary through merging, while Unigram starts with many possible pieces and removes less useful ones.

Byte-Level Tokenization

Byte-level tokenization works with the bytes used by computers to represent text.

Because virtually all digital text can be represented as bytes, byte-level approaches can handle a very wide range of languages, symbols, emojis, and unusual characters.

Modern tokenizers may combine byte-level representation with algorithms such as BPE.

The main advantage is that the tokenizer can represent almost any input without depending entirely on a fixed collection of ordinary characters or words.

Important Tokenizer Concepts

Q1. What is a Vocabulary in Tokenization?

A **vocabulary** is the collection of tokens recognized by a tokenizer. Every token in the vocabulary is assigned a unique numerical ID called a **token ID**.

A simplified vocabulary might contain:

"the" → 25

"AI" → 854

"learn" → 1245

"ing" → 372

The number of tokens in the vocabulary is called the **vocabulary size**.

The tokenizer uses this vocabulary to convert text into token IDs. The Transformer model then uses those IDs to find the corresponding learned embeddings.

An important point is that the tokenizer and the pretrained model are closely connected. The model's embedding table was trained according to the tokenizer's vocabulary and token IDs. Therefore, we generally cannot randomly replace the tokenizer of a pretrained model and expect the model to work correctly.

A simple way to remember this is:

Vocabulary = Collection of known tokens

Token ID = Unique numerical ID assigned to a token

Q2. What are Special Tokens?

Special tokens are tokens that perform specific functions rather than representing ordinary words.

Different models use different special tokens.

Common examples include:

[PAD] — Padding token: Used to make shorter sequences the same length as longer sequences in a batch.

[UNK] — Unknown token: Used by some tokenizers when text cannot be represented using known vocabulary tokens.

[MASK] — Mask token: Used in Masked Language Modeling, especially with models such as BERT.

BOS — Beginning of Sequence: Indicates the beginning of a sequence.

EOS — End of Sequence: Indicates the end of a sequence.

BERT also commonly uses **[CLS]** and **[SEP]**. [CLS] is placed at the beginning of an input and its resulting representation is commonly used for sequence-level tasks. [SEP] is used to mark boundaries or separate sequences.

The exact special tokens and their purposes depend on the model and tokenizer, so they should always be interpreted according to that model's configuration.

Q3. What are Padding and Truncation?

Padding and **truncation** are important techniques used to control sequence length.

Suppose we want to process two sentences together:

Sentence 1: "I like AI."

Sentence 2: "I am currently learning about Transformer models."

After tokenization, the second sentence will probably contain more tokens than the first.

When multiple sequences are processed together as a **batch**, padding can add special [PAD] tokens to shorter sequences so that the sequences have compatible lengths.

For example:

Sequence 1: [25, 48, 72, PAD, PAD]

Sequence 2: [16, 38, 51, 94, 103]

An **attention mask** can then tell the model which positions contain real tokens and which positions are padding.

For example:

Input IDs:

[25, 48, 72, PAD, PAD]

Attention Mask:

[1, 1, 1, 0, 0]

Conceptually, 1 represents a position that should be considered and 0 represents padding that should normally be ignored.

Truncation solves the opposite problem. If a tokenized sequence is longer than the allowed or configured maximum length, truncation removes tokens so that the sequence fits.

Therefore:

Padding → Adds tokens to shorter sequences

Truncation → Removes tokens from sequences that are too long

Both are concepts you will encounter frequently when using Hugging Face tokenizers.

Unigram starts with many possible token pieces and learns which ones are most useful.

Byte-Level approaches represent text using low-level byte units and can handle a very wide range of text.

The tokenizer prepares the text, while the Transformer model performs the actual neural-network processing.